



4주차 세션 정리

Status	In progress
Task type	

Attention → Transformer → BERT

1 Transformer 이전 흐름 정리

◆ Sequential Models (RNN 계열 이전)

✓ 언어의 특징

- 단어 순서가 매우 중요
- 이전 단어를 기반으로 다음 단어 예측

◆ n-gram (초기 통계 기반 모델)

- 이전 n개의 단어 기반 확률 계산
- 단어 빈도 기반 확률 추정

✗ 한계

- 데이터 희소 문제
- "extremely"와 "really"를 다르게 취급
- 긴 문맥 반영 어려움

◆ NNLM (Neural Network Language Model)

- 단어를 **embedding vector**로 표현
- 의미가 유사한 단어끼리 벡터 공간에서 가까움

✗ 한계

- 고정된 길이 입력만 가능
-

◆ RNN

- hidden state를 통해 과거 정보를 누적
- 입력 길이 제한 없음

✗ 한계

- 장기 의존성 문제 (Long-term dependency)
 - 순차 처리 → 병렬 처리 불가능
-

◆ LSTM & GRU

- 게이트 구조 도입 (input, forget, output 등)
- 장기 의존성 완화

✗ 여전히 순차적 처리

→ 연산 느림

→ 필요한 정보만 선택적으로 보기 어려움

2 Seq2Seq

◆ 구조

Encoder → Decoder

- Encoder: 입력 문장 압축
 - Decoder: 출력 문장 생성 (+ <SOS> 토큰)
-

◆ Teacher Forcing

- 학습 시 정답(GT)을 다음 입력으로 사용
 - 안정적인 학습 가능
-

◆ Context Vector

- Encoder의 마지막 hidden state를 전달
- 입력 길이 ≠ 출력 길이 가능

✗ 한계

- 하나의 context vector에 모든 정보 압축
→ 정보 병목(Bottleneck) 문제 발생
-

3 Attention

◆ 핵심 아이디어

| 모든 hidden state를 Decoder로 전달하자

◆ 작동 과정

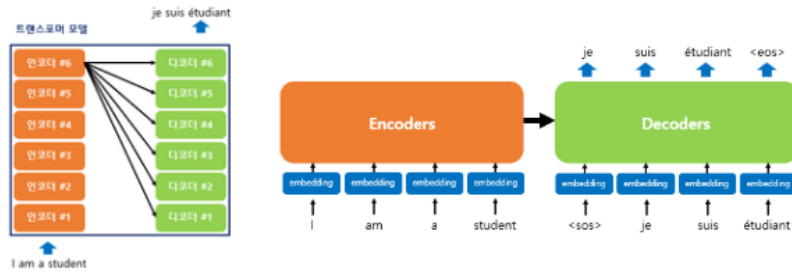
1. 각 Encoder hidden state 계산
 2. Decoder 현재 상태와 비교 → Attention score 계산
 3. Softmax → 중요도 확률화
 4. 가중합 → Context Vector 생성
-

✓ 차이점

- Seq2Seq: context vector 1개
 - Attention: **단어마다 다른 context vector 생성**
-

4 Transformer

TRANSFORMER



◆ 구조 특징

- Encoder-Decoder 구조 유지
- 여러 Layer를 Stack
- 완전 병렬 처리
- RNN 제거

What is Attention

1 입력 → Embedding

- 문장을 입력하면 각 단어를 **embedding vector**로 변환
- 각 토큰은 고정 차원의 벡터로 표현됨

2 Self-Attention

- 각 토큰을 **Query (Q), Key (K), Value (V)** 벡터로 변환
- Query와 Key의 유사도를 계산하여 Attention Score 생성
- Softmax를 통해 가중치(확률)로 정규화

- 해당 가중치를 Value에 곱해 가중합 → **Attention Output 생성**

👉 즉, 각 단어가 문장 내 다른 단어들과 얼마나 관련 있는지를 계산하는 과정

3 Multi-Head Attention

- 하나의 Attention만 사용하면 한 가지 관계만 학습할 수 있음
- 여러 개의 head를 사용하여 서로 다른 관점 학습

예:

- 문법적 관계
- 의미적 유사성
- 장거리 의존 관계

👉 각 head가 다른 특징을 학습하여 문장을 다각도로 이해

4 Decoder Mask (Look-ahead Mask)

- 디코더에서 미래 토큰을 보지 못하도록 제한
 - 아직 생성되지 않은 단어에 attention하지 못하게 하여 **치팅 방지**
-

5 Feed-Forward Network (FFN)

- 각 토큰에 독립적으로 적용되는 비선형 변환
- 표현력을 증가시켜 더 복잡한 특징 학습 가능

구조:

- Linear → ReLU (또는 GELU) → Linear
-

6 Residual Connection + Layer Normalization

- Residual Connection: 입력을 출력에 더해 정보 손실 방지
- Layer Normalization: 분포를 정규화하여 학습 안정화

👉 깊은 네트워크에서도 안정적으로 학습 가능

5 BERT

◆ 특징

- Transformer의 **Encoder**만 사용
 - 양방향 문맥 이해
 - 사전학습 + 파인튜닝 구조
-

◆ BERT 입력 구조

1 Original Text

2 Tokenization (WordPiece 등)

3 Special Token 추가

- [CLS] : 문장 대표 벡터
 - [SEP] : 문장 구분
-

◆ Embedding 구성

- Token Embedding
- Segment Embedding
- Position Embedding

→ 합산 후 Encoder 입력

◆ Pre-Training

1 MLM (Masked Language Model)

- 일부 단어를 [MASK]로 가림
 - 가려진 단어 예측
 - 양방향 문맥 학습 가능
-

2 NSP (Next Sentence Prediction)

- 두 문장이 연속된 문장인지 이진 분류
 - 문장 관계 학습
-

◆ Fine-Tuning

사전학습된 BERT 위에

간단한 Classification Layer 추가

활용 예:

- 감정 분석
 - 문장 분류
 - 질의응답
 - 개체명 인식
-

◆ 특징

- Transformer의 **Encoder**만 사용
 - 양방향 문맥 이해
 - 사전학습 + 파인튜닝 구조
-

◆ BERT 입력 구조

1 Original Text

2 Tokenization (WordPiece 등)

3 Special Token 추가

- [CLS] : 문장 대표 벡터
 - [SEP] : 문장 구분
-

◆ Embedding 구성

- Token Embedding
- Segment Embedding
- Position Embedding

→ 합산 후 Encoder 입력

◆ Pre-Training

1 MLM (Masked Language Model)

- 일부 단어를 [MASK]로 가림
 - 가려진 단어 예측
 - 양방향 문맥 학습 가능
-

2 NSP (Next Sentence Prediction)

- 두 문장이 연속된 문장인지 이진 분류
 - 문장 관계 학습
-

◆ Fine-Tuning

사전학습된 BERT 위에

간단한 Classification Layer 추가

활용 예:

- 감정 분석

- 문장 분류
- 질의응답
- 개체명 인식